

Applied Focus: Imperfect-Information AI Systems

Applied Focus: Imperfect-Information AI Systems

Public Edition

Author: Mauricio Garcia Villanueva

Project: Liar's Dice CFR Lab

Live demo: emgarviii.github.io/liars_dice/

Purpose

This applied focus paper connects the Liar's Dice CFR Lab to the broader field of imperfect-information game solving. The original course deliverable explored concepts like Nash equilibrium, minimax reasoning, Counterfactual Regret Minimization, poker AI, and multi-agent learning. This public edition keeps that direction, but cleans up the structure and aligns the claims with the current validated demo.

The core idea is simple: many important decisions happen when agents do not see the full state of the world. In games, that means hidden cards or dice. In applied systems, it can mean incomplete market information, uncertain counterparties, adversarial incentives, or delayed feedback. The project uses a simplified dice game as a small, inspectable environment for those ideas.

Imperfect Information

An imperfect-information game is a game where at least one player must act without seeing the complete state. Poker is the standard example because each player has private cards. Liar's Dice has the same kind of structure because each player has private dice.

The important part is not the theme of the game. The important part is the decision problem:

What do I know?

What does the other player know?

What can the other player infer?

Which action is best when the missing information changes the value of every choice?

This is why imperfect-information games are useful for AI research. They force an agent to reason over beliefs and incentives, not just visible board positions.

Multi-Agent Learning

Multi-agent learning is different from ordinary supervised learning because the environment includes other decision makers. If one agent changes strategy, the other agent's best response may change too. That creates a moving target.

In the Liar's Dice lab, the AI is not learning from a fixed dataset of labeled examples. It trains through simulated interaction. The claimant policy learns which claims are useful under hidden information. The responder policy learns when to believe and when to challenge.

That self-play framing is one reason the project maps well to AI/ML interviews. It shows the difference between fitting a static dataset and designing a decision system inside an adversarial environment.

Game-Solving Concepts

Nash equilibrium is a strategy profile where no player can improve by unilaterally changing strategy. It is a central concept in game theory, but it is also easy to overclaim. A project can use CFR-style methods without proving that the final policy is a Nash equilibrium.

Minimax reasoning is another core idea. In zero-sum settings, each player can think in terms of maximizing their own payoff while the opponent tries to minimize it. That framing is powerful, but many practical games and multi-agent systems are more complicated than a clean two-player zero-sum example.

Counterfactual Regret Minimization is one of the most important algorithmic ideas for extensive-form imperfect-information games. CFR repeatedly updates a policy by comparing chosen actions against alternative actions that could have been taken at the same information set. Over time, regret minimization can produce strong average strategies in the right settings.

The public demo uses CFR-style training on a simplified game abstraction. That wording is intentional. The current artifact demonstrates the pipeline and measured behavior, but it does not claim a formal equilibrium proof.

Why Poker AI Matters

Poker AI is the best-known applied example of imperfect-information game solving. Systems like Libratus and Pluribus showed that self-play, abstraction, and search can produce strategies strong enough to compete with expert humans. Those systems also made clear that practical game solving is not just one algorithm. It is a combination of abstraction, offline training, evaluation, and real-time or targeted refinement.

The Liar's Dice project is much smaller, but the structure is similar in spirit:

reduce the game to a tractable representation,

train policies offline,

evaluate against opponent classes,

inspect where the policy fails,

explain what the result does and does not prove.

That structure is the applied lesson. I am not claiming that this project is comparable in scale to poker AI systems. I am showing that I understand the engineering pattern at a smaller scale and can build a public artifact around it.

Why Liar's Dice Works as a Teaching Environment

Liar's Dice is easy to explain. Each player rolls dice privately. A claim is made about the total number of dice showing a face. The responder has to decide whether the claim is likely enough to believe.

That makes it useful for communicating hidden information. A visitor can immediately see their own dice and the hidden AI dice. If the claim is "three of face four," the visitor has to combine visible evidence with uncertainty about the opponent's hand.

The simplified public mode removes repeated raising so the decision is easier to isolate. That choice makes the project more understandable as an AI/ML demo, even though it means the solver is not solving the full classic game.

Applied Relevance

The project has relevance beyond games because the same decision pattern appears in other domains:

A trader acts without seeing every other participant's position.

A fraud or risk system must infer intent from partial evidence.

A negotiation agent must reason about hidden preferences.

A market-making system faces adversarial selection and uncertainty.

A security model must perform well against adaptive behavior, not just fixed tests.

The Liar's Dice lab is not a finance model and should not be presented as one. The connection is conceptual: hidden state, strategic agents, probabilistic policies, and adversarial evaluation.

That is why I think the project fits AI/ML and quant-style conversations. It demonstrates the kind of reasoning those areas require, but in a compact environment where the rules and metrics are visible.

Evaluation as the Main Applied Lesson

The strongest part of the public project is the evaluation story. The updated count-aware policy improved benchmark average win rate from 45.3 percent to 73.0 percent across seeded opponent profiles. That is useful evidence that the policy became better against the benchmark suite.

But benchmark success is not the same as robustness. The best-response-style diagnostic still found exploitable pressure, especially on the claimant side. That matters because in adversarial systems, a weak opponent profile can make a policy look better than it really is.

The public results page makes this distinction explicit. It explains what the win rates mean, what opponents they are against, and why they do not prove equilibrium.

Public Framing

The right way to describe the project is:

This is an end-to-end imperfect-information game-solving lab built around a simplified Liar's Dice challenge game. It includes Python CFR-style self-play, policy export, deterministic evaluation, a playable TypeScript demo, a classic rules comparison, and a curated paper archive.

The wrong way to describe it would be to present it as a classic-game solver.

Another overclaim would be to present it as mathematically certified optimal play.

The public edition avoids those overclaims because the measured evidence is more interesting than hype. The policy improved, but it remains exploitable. That is a real research result.

Takeaways

Imperfect-information games are useful because they force agents to act under hidden state.

Multi-agent learning is useful because the opponent is part of the environment.

CFR-style training is useful because regret minimization gives a principled way to update policies over repeated self-play.

Evaluation is necessary because benchmark win rates can hide exploitability.

The Liar's Dice CFR Lab is valuable as a portfolio artifact because it connects these ideas into one public, reproducible system.